

Learning Representations for Multivariate Time Series Classification via Context Prediction

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Time series classification is critical across domains such as healthcare, robotics,
2 fitness, and industrial monitoring. Modern time series classification relies on learn-
3 ing informative representations, but obtaining sufficient labeled data to learn such
4 representations remains costly and often impractical. Self-supervised representa-
5 tion learning, especially contrastive learning and masked modelling, offers a
6 promising path to learning useful representations directly from unlabelled data.
7 However, existing methods rely on carefully designed augmentation strategies or
8 complex masking and reconstruction objectives that introduce domain-specific
9 biases and computational overhead. In this work, we show that a simple pretext
10 task based on context prediction is sufficient to learn meaningful representations for
11 multivariate time series classification. Given a reference patch from a time series
12 instance, our method, C-Pred, trains the encoder to identify which of K candidate
13 patches from the same instance is the immediate temporal neighbour, a K -way
14 classification task that requires no data augmentation, no cross-instance negative
15 sampling, and no reconstruction. The difficulty of the pretext task is controlled
16 by K , providing a principled mechanism to balance upstream task tractability and
17 downstream representation quality. C-Pred achieves competitive performance on
18 28 UEA multivariate benchmarks and six large-scale real-world datasets (>20k
19 instances) while substantially reducing training time compared to contrastive and
20 masked modelling baselines.

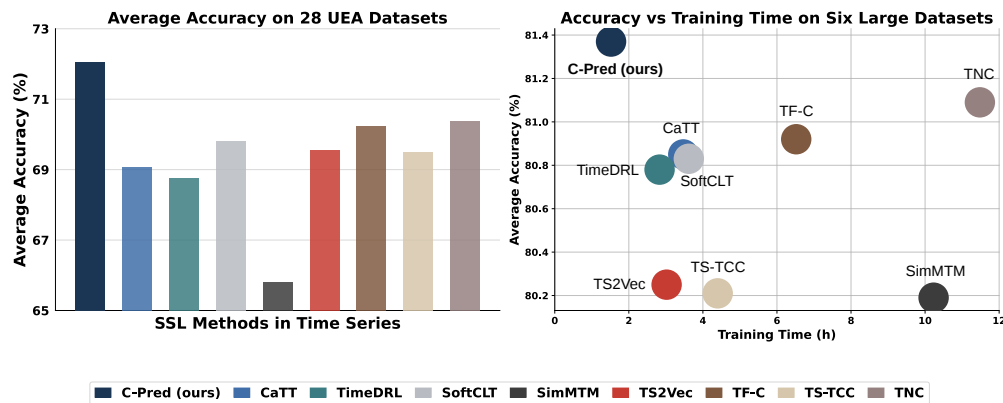


Figure 1: Training efficiency and downstream performance of SSL methods. *Left*: Average accuracy across 28 UEA benchmark datasets. *Right*: Average accuracy vs total training time across six large-scale datasets (>20k samples).

1 Introduction

Time series data is ubiquitous across a wide range of real-world domains, including healthcare (diagnosis and patient monitoring), robotics, industrial process monitoring, and wearable-based activity recognition. In all these settings, the ability to accurately classify time series signals is of direct practical importance, yet the scarcity of expert-annotated labels remains a persistent bottleneck that limits the applicability of fully supervised approaches. Self-supervised learning (SSL) has emerged as a compelling alternative, enabling models to learn rich and transferable feature representations directly from unlabelled data. By designing pretext tasks that exploit the inherent structure of the data, SSL methods pre-train an encoder whose representations can subsequently improve downstream classification performance under limited supervision.

Two families of SSL methods have proven particularly effective for time series representation learning. *Contrastive learning* methods [Eldele et al., 2021, Yue et al., 2022, Tonekaboni et al., 2021, Zhang et al., 2022, Lee et al., 2024] learn representations by pulling together embeddings of semantically similar views (positive pairs) while pushing apart dissimilar ones (negative pairs). *Masked modelling* methods [Zerveas et al., 2021, Dong et al., 2023, Nie et al., 2023] mask portions of the input and train the encoder to reconstruct the missing content, encouraging the model to capture the underlying temporal structure. Both paradigms have demonstrated strong empirical performance and have advanced the state of the art on several time series benchmarks.

However, these approaches have notable limitations. Contrastive learning depends heavily on the construction of positive and negative pairs. Augmentation-based methods may distort the semantic structure of the signal, while temporal adjacency methods can fail on periodic signals where distant segments are more similar than neighbouring ones, leading to false negatives and noisy supervision. Masked modelling avoids augmentations but introduces other challenges, including sensitive masking strategies, higher computational cost, and reconstruction objectives that may overemphasise low-level signal details instead of meaningful semantic representations.

In this work, we propose *C-Pred*, a simple and efficient SSL framework for multivariate time series classification based on a context prediction pretext task. Rather than constructing positive and negative pairs or learning to reconstruct masked inputs, C-Pred formulates the SSL objective as a *K-way classification task*: given a time series instance partitioned into P non-overlapping patches of length L , a reference patch i is selected, and the encoder must identify which of K candidate patches drawn from the same instance is the immediate temporal neighbour of the reference. This admits a simple and tractable pretext task that has exactly one correct answer per instance.

This formulation offers three advantages over existing approaches. **First, no positive pair selection is required.** C-Pred avoids both augmentation-based and adjacency-based contrastive learning by defining the task directly from the temporal structure of each instance, eliminating assumptions about semantic-preserving transformations and reducing false negatives in periodic signals. **Second, no masking or reconstruction is needed.** The task is formulated as a simple K -way classification problem, avoiding decoders, reconstruction losses, and masking strategies. **Third, C-Pred is computationally efficient.** Instead of processing the full sequence or multiple augmented views, the encoder operates only on a reference patch and K candidate patches, substantially reducing computation, particularly in the binary setting ($K = 2$).

We evaluate C-Pred on six large-scale real-world benchmarks with more than 20k instances each, and on 28 UEA multivariate archive spanning diverse temporal domains. C-Pred achieves competitive performance across all settings while maintaining substantially reduced training time compared to contrastive and masked modelling baselines.

Our main contributions are summarised as follows:

- We propose C-Pred, a pretext task for self-supervised multivariate time series representation learning based on next-patch identification via a K -way classification objective, requiring no positive pair selection, no cross-instance negative sampling, and no masking or reconstruction.
- We provide empirical analysis of the effect of channel independence, channel mixing, encoder architecture, and prediction head, on the speed and downstream classification performance on multivariate time series.

- We evaluate C-Pred on six large-scale real-world benchmarks and 28 UEA multivariate datasets, demonstrating strong and consistent performance across diverse temporal dynamics with substantially reduced training time compared to existing SSL methods.

2 Related Work

Self-Supervised Learning. Self-supervised learning (SSL) learns meaningful representations from unlabelled data by defining pretext tasks that exploit the inherent structure of the data. Early approaches formulated these as proxy objectives such as predicting image rotations [Gidaris et al., 2018], solving jigsaw puzzles [Noroozi and Favaro, 2016], or inferring spatial context Doersch et al. [2015], encouraging models to capture semantic structure without manual annotations. However, these methods often suffered from a mismatch between the pretext objective and downstream tasks, leading to representations that overfit to the proxy task. Contrastive learning addressed this limitation by replacing fixed proxy tasks with a flexible objective that directly encodes desired invariances into the representation space. SimCLR [Chen et al., 2020] demonstrated that strong data augmentation combined with a normalised temperature-scaled cross-entropy loss is sufficient to learn highly transferable visual representations. MoCo [He et al., 2020] introduced a momentum encoder and memory bank to decouple batch size from the number of negatives, improving scalability. DINO [Caron et al., 2021] extended this to a self-distillation setting without explicit negatives, using a teacher-student architecture to prevent representation collapse. Masked image modelling subsequently emerged as a complementary paradigm, MAE [He et al., 2022] demonstrated that reconstructing masked patches of an image from visible context is sufficient to learn rich and transferable visual representations. These works collectively established that the choice of SSL objective and the structure of the supervisory signal critically determine representation quality, a finding that directly motivates our work.

Self-Supervised Learning in Time Series. Motivated by advances in CV and NLP, a growing body of work has adapted SSL to the time series domain, primarily through contrastive learning and masked modelling. TNC [Tonekaboni et al., 2021] exploits temporal neighbourhood structure by treating signals from nearby time windows as positives and distant windows as negatives. TS-TCC [Eldele et al., 2021] combines temporal and contextual contrasting with weak and strong augmentations to learn representations robust to temporal perturbations. TF-C [Zhang et al., 2022] aligns representations in both the time and frequency domains through cross-domain contrastive learning. TS2Vec [Yue et al., 2022] introduces a hierarchical contrastive framework operating at both the instance and temporal level using randomly masked overlapping windows, producing dense timestep-level representations. SoftCLT [Lee et al., 2024] relaxes the hard positive/negative assignment by introducing soft similarity labels based on temporal distance. TimeDRL [Chang et al., 2024] proposes a disentangled representation learning framework that jointly optimizes timestamp-level predictive objectives and instance-level contrastive objectives. CaTT [Shamba et al., 2025] eliminates augmentation and multiple encoder passes by contrasting temporally adjacent representations within a sequence, treating nearby timestamps as positives. On the masked modelling side, SimMTM [Dong et al., 2023] frames SSL as a masked reconstruction task over manifold-aware patch aggregations, recovering masked time points from multiple masked series. While these methods achieve strong empirical performance, they share a common dependency on either carefully designed augmentation strategies, which embed domain-specific assumptions and risk distorting the signal, or complex masking and reconstruction objectives that introduce computational overhead. Our work takes a different approach, formulating the SSL objective as a simple K-way classification pretext task that requires neither augmentation nor reconstruction.

Proxy Objectives via Temporal Context Prediction. In contrast to the dominance of contrastive and reconstruction-based objectives, early proxy-based pretext tasks [Doersch et al., 2015, Noroozi and Favaro, 2016, Pathak et al., 2016, Gidaris et al., 2018], have received limited attention in the time series domain. This is largely because time series SSL emerged after the shift in computer vision toward contrastive learning and masked modeling, leading to a direct adoption of these paradigms without revisiting earlier alternatives. The question on whether these simpler pretext tasks offer advantages in the time series setting remains largely unexplored.

Our work is inspired by the context prediction pretext task of Doersch et al. [2015], which uses spatial relationships between image patches as a supervisory signal for representation learning. However,

128 directly adapting this idea to time series is ineffective, since temporal patches from the same instance
 129 are highly similar, making fine-grained position prediction ambiguous and weakly supervised (Fig. 2).
 130 We instead reformulate the task: given a reference patch and K candidate patches, the model predicts
 131 which candidate is the immediate temporal neighbour. This converts an ill-posed regression problem
 132 into a simple K -way classification task with a unique correct answer. To our knowledge, C-Pred is
 133 the first to systematically adapt context prediction to multivariate time series representation learning
 134 in this form.

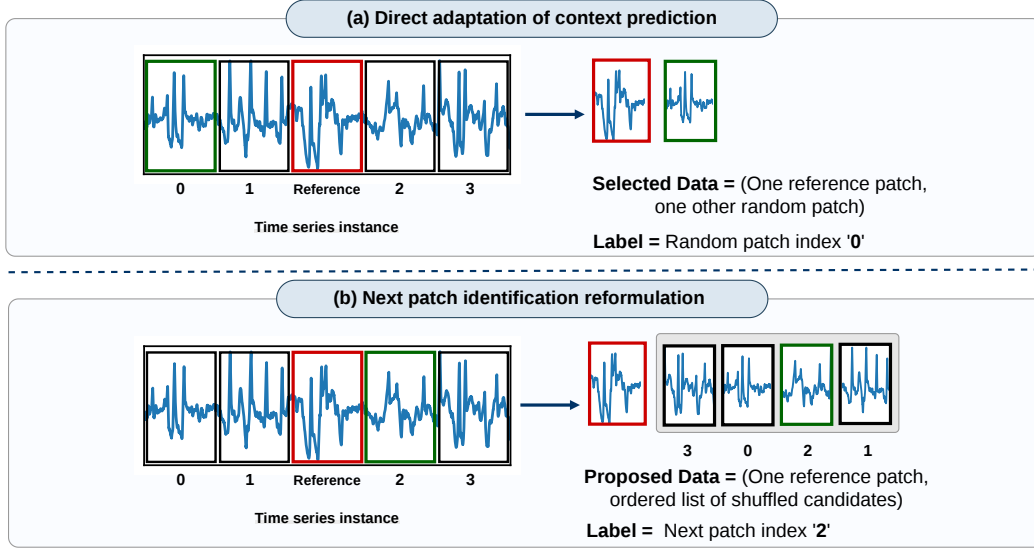


Figure 2: (a) Direct adaptation of image-based context prediction to time series. (b) C-Pred reformulation as next-patch identification via K -way classification.

135 3 C-Pred Method

136 3.1 Problem Definition

137 Let $\mathcal{X} = \{x_1, x_2, \dots, x_N\}$ denote a collection of N unlabelled time series instances, where each
 138 $x_i \in \mathbb{R}^{T \times C}$, with T denoting the sequence length and C the number of channels. Our objective
 139 is to learn an encoder $f_\theta : \mathbb{R}^{L \times C} \rightarrow \mathbb{R}^d$ that maps a patch of length $L < T$ to a d -dimensional
 140 representation, such that the learned embeddings are discriminative and transferable to downstream
 141 classification tasks without any labelled supervision.

142 We adopt a pretext task framework in which the supervisory signal is derived entirely from the
 143 temporal structure of individual instances, without requiring cross-instance negative sampling, data
 144 augmentation, or signal reconstruction. The key design choice, and the central focus of this work, is
 145 the formulation of temporal context prediction as a tractable K -way classification objective.

146 3.2 Why Pretext Tasks for Time Series

147 The design of the self-supervised objective is the most consequential decision in SSL, as it determines
 148 what structure the encoder is required to learn. For the learned representation to support downstream
 149 classification, the pretext task must satisfy three competing constraints.

150 **Label Preservation.** The pretext task must not require the model to learn invariances that destroy
 151 class-relevant information. Augmentation-based contrastive methods risk this when transformations
 152 alter the temporal dynamics that distinguish classes.

153 **Tractability.** The pretext task must be neither too easy nor too hard. A task that is too easy, such as
 154 predicting whether two identical patches are the same, provides no useful learning signal. A task that

is too hard, such as predicting the precise temporal position of a patch among all P possible positions in a long sequence, provides a signal too sparse for the encoder to learn from efficiently.

Signal Density. The pretext task should provide dense supervisory signal per training step. Contrastive methods with a single positive pair per instance provide $2B$ positive assignments per batch update. Masked modelling provides dense signal at the token level but at the cost of a decoder and reconstruction loss. A well-designed pretext task should provide comparable signal density with lower architectural and computational overhead.

These constraints together motivate our reformulation of context prediction for time series.

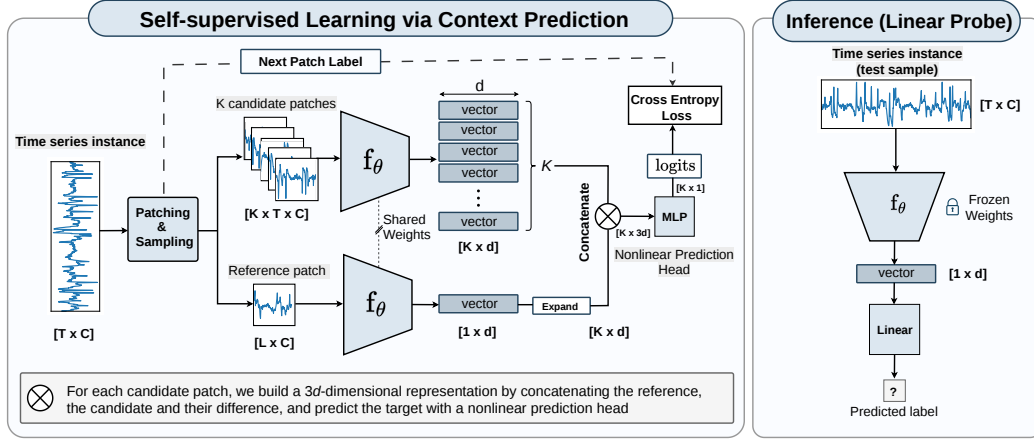


Figure 3: Overview of the proposed C-Pred framework. Given a reference patch, the model learns representations by identifying the temporally adjacent patch from a set of candidate patches via a K -way cross-entropy objective.

3.3 Patch Construction and Context Prediction

Given a time series instance $x \in \mathbb{R}^{T \times C}$, we partition it into P non-overlapping patches of equal length:

$$L = \left\lfloor \frac{T}{P} \right\rfloor, \quad x^{(p)} = x_{(p-1)L:pL} \in \mathbb{R}^{L \times C}, \quad p = 1, \dots, P. \quad (1)$$

From the P patches, we select a reference patch $x^{(r)}$ uniformly at random from $\{x^{(1)}, \dots, x^{(P-1)}\}$, excluding the last patch to ensure a valid right neighbour always exists. The true neighbour is defined as the immediately next patch:

$$x^{(r+1)} \in \mathbb{R}^{L \times C}. \quad (2)$$

We then sample $K - 1$ distractor patches uniformly at random from the remaining $P - 2$ patches $\{x^{(p)} : p \neq r, p \neq r + 1\}$, forming a candidate pool of K patches:

$$\mathcal{C} = \{x^{(r+1)}\} \cup \{x^{(d_1)}, \dots, x^{(d_{K-1})}\}, \quad (3)$$

where $d_1, \dots, d_{K-1}$ are the distractor indices. The candidates are randomly shuffled, and the label $y^* \in \{0, \dots, K - 1\}$ records the position of the true neighbour in the shuffled pool. This shuffling is essential to prevent the model from exploiting position bias in the candidate pool.

3.4 Encoder and Prediction Head

Each patch is encoded independently using a shared encoder f_θ :

$$z^{(r)} = f_\theta(x^{(r)}) \in \mathbb{R}^d, \quad z^{(c_k)} = f_\theta(x^{(c_k)}) \in \mathbb{R}^d, \quad k = 0, \dots, K - 1, \quad (4)$$

176 where $x^{(r)}$ denotes the reference patch and $\{x^{(c_k)}\}_{k=0}^{K-1}$ are the K candidate patches in the shuffled
 177 pool.

178 To model interactions between the reference and candidates, we first expand the reference representa-
 179 tion and construct pairwise feature vectors:

$$\tilde{z}^{(r)} = \text{repeat}(z^{(r)}, K) \in \mathbb{R}^{K \times d}, \quad (5)$$

$$180 \quad h_k = \left[\tilde{z}_k^{(r)} \parallel z^{(c_k)} \parallel \left(\tilde{z}_k^{(r)} - z^{(c_k)} \right) \right] \in \mathbb{R}^{3d}, \quad (6)$$

181 where $\parallel$ denotes concatenation.

182 The resulting matrix $H = [h_0, \dots, h_{K-1}] \in \mathbb{R}^{K \times 3d}$ is passed through a nonlinear prediction head
 183 g_ϕ to produce logits:

$$s_k = g_\phi(h_k), \quad k = 0, \dots, K-1, \quad (7)$$

184 yielding $\mathbf{s} = [s_0, \dots, s_{K-1}] \in \mathbb{R}^K$.

185 These logits define a categorical distribution over the candidate pool, enabling direct K -way classifi-
 186 cation without relying on predefined similarity functions (Figure 3).

187 3.5 Training Objective

188 The pretext task is formulated as a K -way classification problem. Given a reference patch and a
 189 shuffled pool of K candidate patches, the model is trained to identify the true neighbour by assigning
 190 the highest logit to the correct candidate. We optimise the standard cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B} \sum_{b=1}^B \log \frac{\exp(s_{y_b^*}^{(b)})}{\sum_{k=0}^{K-1} \exp(s_k^{(b)})}, \quad (8)$$

191 where B is the batch size, $s_k^{(b)}$ denotes the logit for the k -th candidate in instance b , and y_b^* is the
 192 index of the true neighbour.

193 This objective requires no data augmentation, cross-instance negative sampling, or reconstruction
 194 decoder. The encoder is trained purely through a stable classification signal derived from the temporal
 195 structure within each instance.

196 **Difficulty Control via K .** The number of candidates K directly controls the difficulty of the pretext
 197 task. At $K = 2$, the model must distinguish the true neighbour from a single distractor which renders
 198 an easy binary identification task that provides a weak but stable learning signal. As K increases,
 199 the task becomes progressively harder, requiring finer-grained temporal discrimination. We treat
 200 K as a hyperparameter and study its effect in our ablation analysis. Importantly, K also controls
 201 computational cost: since the encoder processes $K + 1$ patches per instance (one reference and K
 202 candidates), larger K increases compute per update but provides a richer training signal. At $K = 2$,
 203 the encoder processes only $3L$ time steps per update, compared to T for full-sequence methods, a
 204 substantial efficiency gain for long sequences where $L \ll T$.

205 **Inference.** The prediction head is discarded at evaluation time. The encoder f_θ processes the full
 206 input sequence of length T to produce a single representation vector $z \in \mathbb{R}^d$, used directly for
 207 downstream tasks via linear probing. The encoder therefore benefits from the full temporal context at
 208 inference, having been trained on diverse patch-level temporal relationships.

209 4 Experiments

210 4.1 Evaluation Protocol

211 **Datasets.** We evaluate C-Pred on six publicly available large-scale real-world time series datasets:
 212 PAMAP2 [Reiss and Stricker, 2012], WISDM2 [Weiss and Lockhart, 2012], HARTH [Logacjov
 213 et al., 2021], SLEEP [Goldberger et al., 2000], ECG [Moody and Mark, 1983], and SKODA [Zappi
 214 et al., 2012], spanning heterogeneous temporal dynamics from $T = 40$ to $T = 3000$, and input
 215 channels ranging from $C = 1$ to $C = 64$. To further assess generalisation across diverse temporal

domains and dataset scales, we additionally evaluate on 28 UEA [Bagnall et al., 2018] multivariate archive. Full dataset statistics are provided in the Appendix.

Evaluation Protocols. To evaluate the quality of learned representations, we follow the widely adopted linear evaluation protocol [Chen et al., 2020, Yue et al., 2022, Shamba et al., 2025], where a linear classifier is trained on top of the frozen encoder and test accuracy serves as a proxy for representation quality.

Baselines. We compare C-Pred against the following time series self-supervised learning methods: TNC [Tonekaboni et al., 2021], TS-TCC [Eldele et al., 2021], TF-C [Zhang et al., 2022], TS2Vec [Yue et al., 2022], SimMTM [Dong et al., 2023], SoftCLT [Lee et al., 2024], TimeDRL [Chang et al., 2024], and CaTT [Shamba et al., 2025]. All baselines are reproduced using their official implementations under a unified evaluation protocol to ensure fair comparison.

Default Settings. All methods share the same InceptionTime backbone [Ismail Fawaz et al., 2020] and a unified data-loading pipeline with aligned hyperparameters wherever applicable. For C-Pred, the encoder maps each patch of length L to a d -dimensional representation with $d = 256$. We use $P = 10$ patches, $K = 2$ candidates for the large-scale experiments and $K = 5$ for the UEA benchmark. The prediction head is a nonlinear 2-layer MLP and is discarded at evaluation time. For the six large-scale datasets, models are trained for 1500 iterations with batch size 128. Due to memory constraints from long sequence lengths, we set the batch size to 16 for the SLEEP and ECG datasets. For the UEA benchmark, we follow the training protocol of Yue et al. [2022]. All experiments are conducted on NVIDIA V100 GPUs.

4.2 Comparison with State-of-the-Art Time Series SSL methods

We assess the quality of learned representations by freezing the pretrained encoder and training a logistic regression classifier on the extracted embeddings. This linear evaluation protocol provides a direct measure of representation quality, independent of any fine-tuning. Table 1 reports results across all six large-scale datasets.

Table 1: Classification performance across six datasets. Best per dataset in **bold**, second-best in *italic*.

Model	ECG	HARTH	PAMAP2	SKODA	SLEEP	WISDM2	Time (h) ↓	Avg Acc
<i>Random Init.</i>	70.41 $\pm$ 6.09	89.78 $\pm$ 1.32	71.73 $\pm$ 1.75	99.11 $\pm$ 0.12	82.22 $\pm$ 0.66	59.97 $\pm$ 1.42	0.00	78.87
TNC	77.93 $\pm$ 5.04	91.70 $\pm$ 1.81	<i>71.59</i> $\pm$ 2.90	99.00 $\pm$ 0.12	85.14 $\pm$ 0.30	61.16 $\pm$ 1.87	11.48	<i>81.09</i>
TS-TCC	76.66 $\pm$ 2.02	89.29 $\pm$ 2.17	<i>70.42</i> $\pm$ 1.40	98.01 $\pm$ 0.29	85.15 $\pm$ 0.34	61.70 $\pm$ 2.58	4.40	80.21
TF-C	76.02 $\pm$ 2.81	92.24 $\pm$ 1.10	71.30 $\pm$ 1.77	98.23 $\pm$ 0.21	85.18 $\pm$ 0.29	<i>62.54</i> $\pm$ 3.99	6.52	80.92
TS2Vec	76.35 $\pm$ 2.07	90.49 $\pm$ 2.67	<i>70.74</i> $\pm$ 0.80	98.93 $\pm$ 0.16	85.05 $\pm$ 0.17	59.95 $\pm$ 2.20	3.02	80.25
SimMTM	76.97 $\pm$ 2.44	89.99 $\pm$ 2.00	70.15 $\pm$ 0.50	98.19 $\pm$ 0.10	85.22 $\pm$ 0.21	60.64 $\pm$ 3.66	10.23	80.19
SoftCLT	76.90 $\pm$ 4.59	91.58 $\pm$ 0.91	70.67 $\pm$ 0.39	99.02 $\pm$ 0.19	85.05 $\pm$ 0.16	61.73 $\pm$ 3.17	3.62	80.83
TimeDRL	<i>78.39</i> $\pm$ 5.12	87.54 $\pm$ 5.25	70.61 $\pm$ 1.10	98.69 $\pm$ 0.14	84.22 $\pm$ 0.57	65.20 $\pm$ 2.05	<i>2.83</i>	80.78
CaTT	78.81 $\pm$ 9.30	<i>93.13</i> $\pm$ 0.53	69.86 $\pm$ 0.33	94.87 $\pm$ 0.69	<i>85.17</i> $\pm$ 0.33	63.25 $\pm$ 3.36	3.47	80.85
C-Pred (ours)	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	<i>99.08</i> $\pm$ 0.12	85.15 $\pm$ 0.17	<i>63.87</i> $\pm$ 4.21	1.52	81.37
C-Pred (<i>channel ind.</i>)	79.88 $\pm$ 2.63	94.45 $\pm$ 0.78	80.95 $\pm$ 0.50	99.62 $\pm$ 0.06	85.15 $\pm$ 0.17	66.09 $\pm$ 1.69	10.76	84.36

Large-Scale Benchmarks. Table 1 reports results across all six large-scale datasets. C-Pred achieves the highest average accuracy of **81.37%**, while requiring only 1.52 hours of training, the lowest among all methods and $7.5\times$ faster than the second best method TNC (11.48 h). On SKODA and PAMAP2, Random Initialisation achieves the highest accuracy suggesting that these datasets are sufficiently structured that even unlearned features provide strong signal, leaving limited room for SSL methods to improve.

A striking finding emerges from an additional ablation reported in the same table: applying C-Pred under *channel independence*, where each channel is encoded separately by the same univariate encoder and representations are aggregated, yields an average accuracy of **84.36%**, a gain of 2.99 percentage points (pp) over channel mixing and surpassing all baselines by a substantial margin across every dataset. Although, the training time under channel independence increases to 10.76 hours, reflecting the cost of processing each channel independently, nevertheless this remains comparable to TNC (11.48 h) while achieving 3.27 percentage points higher average accuracy.

UEA Multivariate Archive. Table 2 reports summary statistics across 28 UEA datasets. C-Pred under channel mixing achieves the best average accuracy (**72.05%**) and the most first-place wins (4),

Table 2: Summary of classification results on the 28 UEA multivariate time series datasets. Best results in **bold**, second-best in *italic*.

Method	Avg. Acc. $\uparrow$	Avg. Rank $\downarrow$	Ranks 1 st $\uparrow$	Avg. Diff. (%) $\downarrow$	W/T/L	Wilcoxon p -value $\downarrow$	Time (h) $\downarrow$
<i>Random Init.</i>	67.28	7.02	0	4.78	17/6/5	0.00	0.00
TNC	70.38	5.59	<u>3</u>	1.68	13/3/12	0.15	0.61
TS-TCC	69.49	6.48	0	2.56	16/4/8	0.00	0.47
TF-C	<i>70.23</i>	<i>5.39</i>	0	<i>1.83</i>	16/2/10	<i>0.03</i>	0.53
TS2Vec	68.55	6.32	1	3.50	18/3/7	0.00	0.37
SimMTM	65.82	7.89	1	6.23	21/1/6	0.00	1.24
SoftCLT	69.61	5.82	1	2.44	15/6/7	0.01	0.25
TimeDRL	68.76	6.00	2	3.29	15/3/10	0.02	0.32
CaTT	69.07	6.29	2	2.99	15/2/11	0.05	0.18
C-Pred (ours)	72.05	4.80	4	—	—	—	<i>0.19</i>
C-Pred (<i>channel ind.</i>)	70.67	4.39	9	1.39	10/5/13	0.46	0.63

confirming strong generalisation across diverse multivariate temporal domains. Channel independence achieves 70.67% average accuracy and a better average rank (4.39 vs 4.80) with nine first-place wins, more than any other method, but a lower overall average due to underperformance on a subset of small datasets where cross-variable dependencies are informative. Wilcoxon signed-rank tests confirm that C-Pred significantly outperforms all methods except TNC ($p = 0.15$), which is the only baseline whose performance is statistically indistinguishable from C-Pred, while requiring a fraction of the training time of TNC. Specifically, TNC trains for 0.61 hours on UEA, more than $3\times$ longer than C-Pred (0.19 h), and achieves a higher average rank (5.59 vs 4.80), a worse W/T/L record, and a higher average accuracy gap. These results collectively demonstrate that C-Pred learns representations that generalise consistently across diverse multivariate temporal domains and achieves the best accuracy-efficiency trade-off among all methods evaluated.

4.3 Computational Efficiency

A key design property of C-Pred is its computational efficiency, which arises directly from the patch-based pretext task formulation. Figure 1, Table 1 and Table 2 report total training time across the six large-scale datasets and the UEA benchmark. C-Pred achieves the lowest total training time overall ($1.52 + 0.19$ h). This efficiency gain comes from the design of the method. At each training step, C-Pred’s encoder processes only $(K + 1)$ patches of length $L = T/P$, a total of $(K + 1)T/P$ time steps, compared to T or $2T$ for single-view and dual-view full-sequence methods. For the large scale datasets, at the default settings of $P = 10$ and $K = 2$, the encoder sees only $3T/10 = 0.3T$ time steps per update, a 70% reduction in encoder compute relative to full-sequence methods. This reduction scales proportionally with sequence length, making C-Pred particularly efficient for long sequences such as ECG ($T = 2500$) and SLEEP ($T = 3000$).

5 Analysis

Table 3 reports a comprehensive ablation study of C-Pred across six design dimensions on the large-scale benchmarks and the 28 UEA datasets. Default settings are highlighted in gray.

Pretext Task Formulation. We compare our predictive K -way cross-entropy objective against a contrastive alternative that uses the same patch pairs but optimises NT-Xent instead. The contrastive variant achieves higher average accuracy on the large-scale benchmarks but lower accuracy on UEA, suggesting that the contrastive objective overfits to the characteristics of large datasets while the predictive objective generalises more consistently across diverse dataset scales. We adopt the predictive formulation as the default for its more stable cross-entropy objective and superior UEA performance.

Prediction Head. A nonlinear head marginally outperforms a linear head on UEA (72.05% vs 69.54%) with negligible difference on the large-scale benchmarks. This is consistent with findings in

Table 3: Ablation study of C-Pred across different settings on large-scale datasets and the 28 UEA benchmark.

Method	ECG	HARTH	PAMAP2	SKODA	WISDM2	28 UEA Datasets	Total Time (h) $\downarrow$
<i>Classification Logits</i>							
Contrastive	81.07 $\pm$ 6.19	93.47 $\pm$ 0.87	70.70 $\pm$ 0.29	99.51 $\pm$ 0.11	65.02 $\pm$ 3.23	69.56	1.52
Predictive	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	72.05	1.55
<i>Prediction Head</i>							
Linear	76.67 $\pm$ 7.66	92.51 $\pm$ 1.04	70.31 $\pm$ 0.62	99.01 $\pm$ 0.15	64.10 $\pm$ 2.26	69.54	1.51
Nonlinear	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	72.05	1.55
<i>Encoding method</i>							
Channel Independence	79.88 $\pm$ 2.63	94.45 $\pm$ 0.78	80.95 $\pm$ 0.50	99.62 $\pm$ 0.06	66.09 $\pm$ 1.69	70.67	11.14
Channel Mixing	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	72.05	1.55
<i>Neighbor Identification</i>							
Previous Patch	73.77 $\pm$ 3.17	92.95 $\pm$ 1.50	70.44 $\pm$ 0.50	99.00 $\pm$ 0.11	62.04 $\pm$ 2.83	70.70	1.55
Next Patch	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	72.05	1.55
<i>Effect of K</i>							
K = 2	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	70.58	1.53
K = 5	77.50 $\pm$ 4.53	92.98 $\pm$ 2.00	69.99 $\pm$ 0.63	98.99 $\pm$ 0.21	61.88 $\pm$ 2.88	72.05	2.79
<i>Effect of P</i>							
P = 5	75.50 $\pm$ 0.76	91.86 $\pm$ 1.55	70.13 $\pm$ 0.36	98.98 $\pm$ 0.09	63.02 $\pm$ 3.03	70.93	1.76
P = 10	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	72.05	1.55
<i>Encoder</i>							
- FCN	71.46 $\pm$ 3.88	92.56 $\pm$ 1.28	71.63 $\pm$ 0.62	98.37 $\pm$ 0.14	62.15 $\pm$ 1.95	70.24	0.58
- ResNet	72.58 $\pm$ 1.98	91.80 $\pm$ 0.96	70.48 $\pm$ 1.04	99.21 $\pm$ 0.12	59.31 $\pm$ 3.32	71.10	0.92
- InceptionTime	76.15 $\pm$ 2.73	93.43 $\pm$ 1.37	70.54 $\pm$ 0.55	99.08 $\pm$ 0.12	63.87 $\pm$ 4.21	72.05	1.55

visual SSL [Chen et al., 2020] that a nonlinear projection absorbs task-specific invariances and leaves the encoder output more transferable. We adopt the nonlinear head as default.

Neighbour Identification Direction. Predicting the next patch consistently outperforms predicting the previous patch across all settings, with comparable training time. We attribute this to the causal structure of physical and biological time series, where the current state is more predictive of the future than the past is of the present. We adopt next-patch identification as the default.

Effect of K and P . At $K = 2$ vs $K = 5$, we observe different trends across evaluation settings. On the large-scale datasets, $K = 2$ consistently performs as well as or better than $K = 5$ while requiring substantially lower training time (1.53 vs. 2.79 hours). In contrast, on the 28 UEA datasets, $K = 5$ achieves the best overall accuracy. Based on these observations, we use $K = 2$ as the default setting for large-scale experiments and $K = 5$ for the UEA benchmark. For patch count, $P = 10$ outperforms $P = 6$ consistently across all datasets, with comparable training time (1.55 h vs 1.76 h). Fewer patches produce longer patches that reduce the diversity of temporal contexts seen during training. We adopt $P = 10$ as default.

Encoder Architecture. InceptionTime achieves the best performance on most datasets and the highest UEA average (72.05%), while being marginally slower than FCN (1.55 h vs 0.58 h). FCN achieves competitive performance on PAMAP2 but underperforms InceptionTime on all other datasets. ResNet underperforms both on HARTH (91.80%), WISDM2 (59.31%), and UEA (71.10%), indicating that residual connections alone do not confer an advantage in this setting. All subsequent experiments use InceptionTime as default.

Batch Size. Unlike contrastive methods whose training signal depends critically on batch composition, C-Pred’s intra-instance objective is entirely independent of what other instances appear in the batch. Figure 4 confirms this: C-Pred performance is stable across batch sizes $\{32, 64, 128, 256, 512\}$. This batch-size independence is a practical advantage as C-Pred can be trained with small batch sizes without sacrificing representation quality, making it applicable in memory-constrained settings.

Channel Independence vs Channel Mixing. The most significant finding in our ablation concerns the channel encoding strategy. Under *channel mixing*, the default assumption of every SSL baseline in Table 1, all channels are processed jointly by the encoder. Under *channel independence*, each channel is processed separately by the same univariate encoder and representations are aggregated before the prediction head.

Channel independence produces substantial improvement on the large-scale datasets, at the cost of increased training time (10.76 h vs 1.52 h). Despite this overhead, the channel independence variant remains faster than TNC (11.48 h), the second best channel mixing method at 81.09%, while exceeding it by 3.27 pp. On the UEA benchmark, channel independence improves performance on several individual datasets compared to channel mixing (9 wins vs. 4 wins). However, despite these strong dataset-specific gains, the channel mixing variant achieves a higher overall average accuracy across all 28 datasets (72.05% vs. 70.67%), while also being substantially more computationally efficient (0.19 h vs. 0.63 h). The large increase in training time for channel independence arises from per-channel processing, which significantly increases the effective batch size and memory consumption on datasets with high channel dimensionality.

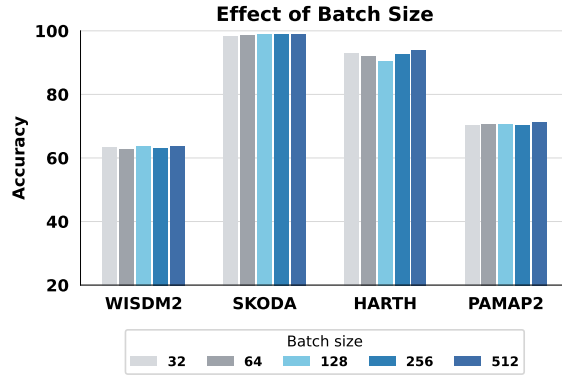


Figure 4: Effect of batch size on linear probing accuracy (%) across four representative large-scale datasets.

6 Conclusion

We present *C-Pred*, a self-supervised framework for multivariate time series classification based on context prediction. Given a reference patch, the encoder predicts its immediate temporal neighbour

from K candidates within the same instance, forming a K -way classification task with a single correct answer and requiring no augmentation, cross-instance negative sampling, or reconstruction. C-Pred achieves competitive results across six large-scale benchmarks and 28 UEA datasets while maintaining the lowest training time among all methods. Our analysis surfaces two findings of broader significance. First, channel independence substantially outperforms channel mixing, despite most existing SSL method for time series classification using channel mixing. Beyond accuracy, channel independence enables a single univariate backbone to handle any channel dimensionality, directly addressing the varying channel count problem that limits scalable time series SSL and foundation model development. Second, C-Pred avoids the overfitting-to-pretext failure that caused proxy objectives to be abandoned in vision using hyperparameter K to provides a principled difficulty control mechanism. Together, these results suggest that the time series SSL community’s focus on contrastive learning and masked modelling, while productive, has overlooked a simpler and more efficient family of objectives. A carefully reformulated context prediction task offers a principled and practical alternative that achieves strong empirical performance without the design complexity of existing approaches. We hope C-Pred serves as both a strong practical baseline and a conceptual argument for revisiting the full spectrum of self-supervised objectives in the time series domain.

Limitations. While C-Pred demonstrates strong performance across a wide range of datasets, its effectiveness depends on the availability of sufficiently discriminative information within local temporal patches. In multivariate time series, this condition is often satisfied, as multiple channels provide complementary signals that make neighboring patches distinguishable. However, in univariate time series with low variability or highly repetitive structure, local patches may be less informative, making the next-patch identification task inherently ambiguous and limiting representation quality.

References

- A. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. Bostrom, P. Southam, and E. Keogh. The uea multivariate time series classification archive, 2018, 2018. URL <https://arxiv.org/abs/1811.00075>.
- M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.
- C. Chang, C.-T. Chan, W.-Y. Wang, W.-C. Peng, and T.-F. Chen. Timedrl: Disentangled representation learning for multivariate time-series. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 625–638. IEEE, 2024.
- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- C. Doersch, A. Gupta, and A. A. Efros. Unsupervised visual representation learning by context prediction. In *Proceedings of the IEEE international conference on computer vision*, pages 1422–1430, 2015.
- J. Dong, H. Wu, H. Zhang, L. Zhang, J. Wang, and M. Long. Simmtm: A simple pre-training framework for masked time-series modeling. *Advances in Neural Information Processing Systems*, 36:29996–30025, 2023.
- E. Eldele, M. Ragab, Z. Chen, M. Wu, C. K. Kwok, X. Li, and C. Guan. Time-series representation learning via temporal and contextual contrasting. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pages 2352–2359, 2021.
- N. M. Foumani, C. W. Tan, G. I. Webb, H. Rezatofighi, and M. Salehi. Series2vec: similarity-based self-supervised representation learning for time series classification. *Data Mining and Knowledge Discovery*, 38(4):2520–2544, 2024.
- S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=S1v4N210->.
- A. Goldberger, L. Amaral, L. Glass, J. Hausdorff, P. Ivanov, R. Mark, J. Mietus, G. Moody, C.-K. Peng, and H. Stanley. Physiobank, physiotoolkit, and physionet : Components of a new research resource for complex physiologic signals. *Circulation*, 101:E215–20, 07 2000. doi: 10.1161/01.CIR.101.23.e215.
- K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9729–9738, 2020.
- K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.
- H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962, 2020.
- S. Lee, T. Park, and K. Lee. Soft contrastive learning for time series. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=pAsQSW1DUf>.
- A. Logacjov, K. Bach, A. Kongsvold, H. B. Bårdstu, and P. J. Mork. Harth: A human activity recognition dataset for machine learning. *Sensors (Basel, Switzerland)*, 21, 2021. URL <https://api.semanticscholar.org/CorpusID:244808329>.

- 418 G. B. Moody and R. G. Mark. A new method for detecting atrial fibrillation using r-r intervals. In
419 *Computers in Cardiology*, pages 227–230, 1983.
- 420 Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term
421 forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- 422 M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles.
423 In *European conference on computer vision*, pages 69–84. Springer, 2016.
- 424 D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning
425 by inpainting. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*,
426 pages 2536–2544, 2016. doi: 10.1109/CVPR.2016.278.
- 427 A. Reiss and D. Stricker. Introducing a new benchmarked dataset for activity monitoring. In *2012*
428 *16th international symposium on wearable computers*, pages 108–109. IEEE, 2012.
- 429 A.-K. Shamba, K. Bach, and G. Taylor. Contrast all the time: Learning time series representation
430 from temporal consistency. In *Proceedings of the European Conference on Artificial Intelligence*
431 *(ECAI)*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 2810–2817,
432 2025. URL <https://doi.org/10.3233/FAIA251137>.
- 433 S. Tonekaboni, D. Eytan, and A. Goldenberg. Unsupervised representation learning for time series
434 with temporal neighborhood coding. In *International Conference on Learning Representations*,
435 2021. URL <https://openreview.net/forum?id=8qDwejCuCN>.
- 436 G. M. Weiss and J. W. Lockhart. The impact of personalization on smartphone-based activity
437 recognition. In *AAAI workshop on activity context representation: techniques and languages*,
438 pages 98–104. Toronto., 2012.
- 439 Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, and B. Xu. Ts2vec: Towards universal
440 representation of time series. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
441 volume 36, pages 8980–8987, 2022.
- 442 P. Zappi, D. Roggen, E. Farella, G. Tröster, and L. Benini. Network-level power-performance trade-
443 off in wearable activity recognition: A dynamic sensor selection approach. *ACM Transactions on*
444 *Embedded Computing Systems (TECS)*, 11(3):1–30, 2012.
- 445 G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff. A transformer-based framework
446 for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD*
447 *conference on knowledge discovery & data mining*, pages 2114–2124, 2021.
- 448 X. Zhang, Z. Zhao, T. Tsiligkaridis, and M. Zitnik. Self-supervised contrastive pre-training for time
449 series via time-frequency consistency. *Advances in Neural Information Processing Systems*, 35:
450 3988–4003, 2022.

451 **Code Availability** – To support reproducibility, the complete codebase and scripts for C-Pred are
452 provided in the supplementary material and will be released publicly upon acceptance.

453 A Reproducibility and Computational Environment

454 To ensure reproducibility, we report all experimental settings and computational details used in our
455 study. Experiments on the large-scale benchmarks are conducted over five random seeds $\{1, 2, 3, 4, 5\}$,
456 and results are reported as averages across seeds. For the UEA benchmarks, we follow prior work
457 and use a fixed random seed of 1. C-Pred uses $K = 2$ for the large scale datasets and $K = 5$ for the
458 UEA benchmark datasets, with $P = 10$ throughout.

459 All experiments are implemented in Python 3.12.3 using PyTorch 2.5.1 with CUDA 12.4. Experiments
460 are executed on NVIDIA Tesla V100S GPUs with 32 GB memory. The system uses NVIDIA driver
461 version 570.172.08.

B Baseline Descriptions.

This appendix provides implementation details and reproduction settings for all baseline methods compared in this work. To ensure a controlled and fair comparison, we standardize the backbone architecture and data-loading pipeline across all methods. Specifically, all models are trained using a shared 1D InceptionTime backbone Ismail Fawaz et al. [2020], ensuring that performance differences reflect contrastive learning objectives rather than architectural choices. All models are optimized using AdamW with weight decay of 3×10^{-4} and momentum parameters $\beta_1 = 0.9$, $\beta_2 = 0.99$, under a cosine learning-rate schedule with linear warm-up during the first 10% of training iterations. Learning rates are selected via a hyperparameter search over $\{1 \times 10^{-3}, 3 \times 10^{-4}, 1 \times 10^{-4}\}$ on the six large-scale datasets. Specifically, for C-Pred, we use learning rates 1×10^{-3} and 3×10^{-4} for the six large-scale and 28 UEA benchmarks respectively. For all methods, we use the official implementation with default method-specific hyperparameters as described in the original work.

C Detailed Dataset Information.

We use publicly available time series datasets previously employed in self-supervised representation learning research. The WISDM2, PAMAP2, SLEEP and SKODA datasets were directly obtained from the repository linked by the Series2Vec project [Foumani et al., 2024], without applying additional preprocessing. For the ECG dataset, we generate random windows for self-supervised learning following Tonekaboni et al. [2021]. Each subject’s continuous 12-hour recording was split into 10-minute segments to reduce redundancy, and during training, a random window of 2,500 time steps was sampled from each segment. The first 5 subjects were held out for testing, with the remaining subjects used for training. This strategy increases variability in the training data while maintaining consistent input dimensions across subjects. For linear probing and kNN evaluation, we extract a deterministic window of 2,500 time steps from the middle of each segment from the train set to ensure reproducible evaluation. For the HARTH dataset, we performed subject-wise split with 22 subjects in total. The first 18 subjects were used for training and the remaining 4 for testing. Each subject’s multivariate time series was split into overlapping windows of 500 time steps with a stride of 250, and the majority class within each window was assigned as the label.

Table 4 summarizes the key characteristics of each dataset. All datasets are split using subject-wise splitting, the SKODA dataset, collected from a single subject, however, does not have subject-wise splits.

Table 4: Summary of datasets used in the large-scale experiments.

Dataset	Train set	Test set	Subject	Sequence Length	Channel	Class
ECG	59,922	16,645	23	2,500	2	4
HARTH	23,025	2,785	22	500	6	18
PAMAP2	51,192	11,590	9	100	52	18
SKODA	22,587	5,646	1	50	64	11
SLEEP	25,612	8,910	20	3,000	1	5
WISDM2	134,614	14,421	29	40	3	6

D Full UEA Benchmarks Results.

Table 5 present the full 28 UEA benchmark results. The UEA archive Bagnall et al. [2018] originally contains 30 datasets. In our experiments, we report results on 28 datasets, excluding *InsectWingbeat* and *EigenWorms*. These two datasets exhibit atypical input shapes and extremely large memory requirements, which lead to runtime or memory errors for several baseline methods despite consistent preprocessing and hyperparameter settings. We note that this behavior is not unique to our evaluation protocol. Prior work such as TS2Vec Yue et al. [2022] similarly omits a comparable subset of datasets, reporting results on 29 UEA datasets, due to analogous issues related to input dimensionality, memory constraints, or training instability.

Table 5: Classification accuracy (%) across 28 UEA multivariate datasets. Best per dataset in **bold**, second-best underlined.

Dataset	CaTT	Rand. Init.	SimMTM	SoftCLT	TF-C	TNC	TS2Vec	TS-TCC	TimeDRL	C-Pred	C-Pred (Ch.I)
ArticularyWordRecognition	<u>98.33</u>	96.33	91.33	97.33	97.33	97.33	97.00	97.33	95.00	97.33	98.67
AtrialFibrillation	26.67	26.67	20.00	<u>33.33</u>	<u>33.33</u>	26.67	20.00	26.67	26.67	40.00	<u>33.33</u>
BasicMotions	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
CharacterTrajectories	99.23	97.98	97.77	99.51	99.51	<u>99.37</u>	99.30	99.30	99.30	99.30	98.82
Cricket	86.11	<u>98.61</u>	88.89	97.22	95.83	100.00	91.67	97.22	<u>98.61</u>	<u>98.61</u>	<u>98.61</u>
DuckDuckGeese	46.00	<u>60.00</u>	54.00	52.00	62.00	54.00	54.00	52.00	50.00	<u>60.00</u>	<u>60.00</u>
ERing	86.30	80.37	89.26	<u>92.22</u>	90.37	88.52	89.63	85.56	92.96	<u>92.22</u>	<u>92.22</u>
Epilepsy	91.30	93.48	94.93	93.48	92.03	<u>95.65</u>	92.03	92.75	94.20	93.48	97.10
EthanolConcentration	27.38	<u>30.80</u>	27.38	28.14	26.24	25.86	30.42	26.24	30.04	25.48	35.36
FaceDetection	52.44	<u>52.67</u>	52.01	52.33	54.60	52.38	52.38	52.89	54.28	52.41	56.84
FingerMovements	60.00	56.00	49.00	56.00	52.00	63.00	47.00	58.00	47.00	56.00	<u>61.00</u>
HandMovementDirection	25.68	35.14	32.43	27.03	32.43	28.38	35.14	32.43	28.38	<u>33.78</u>	<u>33.78</u>
Handwriting	53.29	38.71	21.65	53.41	49.65	53.65	47.18	50.71	58.35	<u>56.00</u>	50.24
Heartbeat	<u>75.12</u>	74.15	73.66	69.27	74.63	73.17	<u>75.12</u>	70.73	74.15	77.07	72.68
JapaneseVowels	<u>98.11</u>	96.49	94.32	95.95	96.76	97.57	95.68	97.03	97.57	97.03	98.65
LSST	50.53	50.49	<u>51.62</u>	50.57	51.34	49.92	50.12	49.76	48.58	49.27	57.34
Libras	88.33	60.56	70.00	80.00	79.44	86.67	77.22	77.78	75.00	81.11	<u>87.78</u>
MotorImagery	<u>56.00</u>	54.00	55.00	44.00	53.00	57.00	54.00	53.00	50.00	<u>56.00</u>	49.00
NATOPS	92.78	83.89	81.11	95.00	88.33	91.11	87.22	85.56	89.44	<u>90.56</u>	<u>93.33</u>
PEMS-SF	75.72	77.46	63.58	79.19	73.99	78.03	80.92	76.30	75.72	80.92	72.25
PenDigits	96.88	97.03	97.88	97.66	97.06	97.03	<u>97.77</u>	97.43	97.74	97.03	97.00
PhonemeSpectra	24.34	19.33	22.85	23.83	<u>26.33</u>	23.50	25.35	24.10	23.59	23.83	26.45
RacketSports	81.58	<u>83.55</u>	79.61	<u>83.55</u>	<u>83.55</u>	79.61	86.18	81.58	76.97	78.29	82.24
SelfRegulationSCP1	78.16	86.01	82.25	82.94	86.69	86.01	80.89	<u>87.03</u>	86.69	86.35	87.37
SelfRegulationSCP2	55.00	48.89	47.22	50.00	52.22	48.89	50.00	46.67	<u>54.44</u>	51.67	45.00
SpokenArabicDigits	97.09	93.36	97.54	97.32	97.04	97.27	97.45	97.00	<u>97.77</u>	97.23	99.00
StandWalkJump	40.00	33.33	33.33	40.00	40.00	40.00	26.67	<u>46.67</u>	20.00	60.00	13.33
UWaveGestureLibrary	71.56	58.44	74.38	77.81	80.62	80.00	79.06	<u>84.06</u>	82.81	86.56	81.25
Average	69.07	67.28	65.82	69.61	70.23	70.38	68.55	69.49	68.76	72.05	<u>70.67</u>

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: We focus mainly on SSL learning for time series classification and the 28 UEA datasets provide us with a way to verify our claim as shown in Table 2

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We mentioned limitations in Section 6

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.

- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This is mainly an empirical paper.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Yes, we provide the full code in the supplementary material, and additional details in Appendix A

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often

one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: We have provided the code in the supplementary material. All datasets used are publicly available and described in Appendix C

Guidelines:

- The answer [\[N/A\]](#) means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [\[No\]](#) is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Yes, we provided this in Appendix C

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Yes, we provided the Wilcoxon p-value test on the 28 UEA in Table 2

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer:[Yes]

Justification: We have provided this in Appendix A

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have read and understood the code of ethics and have made every effort to adhere to it.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: Our work contributes to a better understanding of the time series SSL for classification.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We do not release any new models or datasets. Our work uses standard, publicly available resources.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All codes, dataset and models used in this experiments have been properly credited and cited.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: We have well documented codebase and will make it publicly available upon acceptance

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: This work does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

793 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
794 or other labor should be paid at least the minimum wage in the country of the data
795 collector.

796 **15. Institutional review board (IRB) approvals or equivalent for research with human**
797 **subjects**

798 Question: Does the paper describe potential risks incurred by study participants, whether
799 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
800 approvals (or an equivalent approval/review based on the requirements of your country or
801 institution) were obtained?

802 Answer: [N/A]

803 Justification: This work does not involve human subjects

804 Guidelines:

805 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
806 with human subjects.

807 • Depending on the country in which research is conducted, IRB approval (or equivalent)
808 may be required for any human subjects research. If you obtained IRB approval, you
809 should clearly state this in the paper.

810 • We recognize that the procedures for this may vary significantly between institutions
811 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
812 guidelines for their institution.

813 • For initial submissions, do not include any information that would break anonymity (if
814 applicable), such as the institution conducting the review.

815 **16. Declaration of LLM usage**

816 Question: Does the paper describe the usage of LLMs if it is an important, original, or
817 non-standard component of the core methods in this research? Note that if the LLM is used
818 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
819 scientific rigor, or originality of the research, declaration is not required.

820 Answer: [N/A]

821 Justification: We used an LLM mainly for editing and correcting grammar.

822 Guidelines:

823 • The answer [N/A] means that the core method development in this research does not
824 involve LLMs as any important, original, or non-standard components.

825 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
826 be described.